NIM sizing: from request to recommendation

Implemented API + CLI flow | Current PR behavior | No deployment or reservation

1 Request, authorization and cluster inventory

1 API or CLI request

Model image + exact tag + clusterId
Optional gpuType / --gpu-type
CLI uses the selected cluster context

2 Authorize + validate

Require inference-create permission
Validate cluster, image, tag and type
Rejected requests stop here

3 Read + select inventory

Read every authorized node-pool page
Exact GPU type, or all type/capacity pairs
Empty inventory + all types: 200, results []

Selection nonempty

Continue from right to left

2 Fetch profiles, select a baseline, size the allocation

4 Read cached / NGC data

Exact image/tag cache; expiry: 15 min
Miss / cache failure: query NGC
Cache profiles only; skip empty / errors

5 Match + select profiles

For each selected type/capacity pair
Fit first; direct > tested > GPU-agnostic
Lowest minimum, then profile ID

6 Apply sharing policy

Below 90%: allocate required memory
90% through capacity: one whole GPU
Above capacity: estimated GPU count

Published minimum available

No usable minimum / capacity

HTTP 200 may mix result statuses

3 Return a distinct API outcome

200 recommended

Profile ID, precision, backend, minimum
Memory allocation or GPU count
Evidence + runtime caveats in warning

200 no_recommendation

Missing capacity or usable profile data
Explain why this entry cannot be sized
No suggested allocation for this entry

Request failed

400: invalid input / absent explicit type
401 / 403: access boundary rejected it
503: inventory or NGC retrieval failed

Current boundary

Sizing stops at the response. It does not deploy a model, reserve resources, check quota or automatically prefill a UI.
Installed capacity is not free capacity. Users or a separate integration may apply the suggestion through an existing deploy flow.

Blue: processing Green: successful result Amber: supported alternative / estimate Red: failure Gray: context

API decisions: GPU type, profiles and responses

POST /api/v1/inference/right-sizing/recommendation | Implemented request and response branches

Start with an authorized, valid request

Required: clusterId, image and exact tag. Optional: gpuType. Inventory is read completely before profile selection.
Invalid input -> 400. Missing credentials / denied permission -> 401 / 403 at the access boundary.

1 Select cluster hardware

gpuType omitted or null

Evaluate every distinct reported GPU type/capacity pair.
Empty inventory -> HTTP 200 with results: [].
There is no single global winner across GPU types.

gpuType supplied

Evaluate only the exact inventory GPU type, at each reported capacity.
Requested type absent -> HTTP 400.
Blank or whitespace-only type -> HTTP 400.

2 Fetch the model/tag metadata

Valid, nonempty selection

Read metadata cache / NGC

Exact image/tag cache: default 15 min.
Miss / cache failure -> fetch NGC.
Cache profiles only; inventory stays fresh.

Failure

503 dependency failure

Inventory or NGC retrieval failed.
Includes timeout or malformed data.
No partial recommendation response.

404

NGC 404 / no profiles

Continue with an empty profile set.
This is missing sizing information,
not a dependency failure.

3 Prepare each hardware entry

For each selected GPU type/capacity

Missing or nonpositive capacity -> no_recommendation for this entry.
Resolve the exact PCI identity. Unknown or ambiguous inventory names can still use GPU-agnostic profiles.
An eligible baseline has a profile ID, explicit TP=1 and PP=1, and a positive, finite published minimum.

4 Classify each eligible profile

First matching rule wins; evaluate all candidates

Direct PCI match?

gpu_device equals resolved PCI
Empty tested list is OK

No

Tested PCI match?

Resolved PCI appears in
tested_gpu_devices

No

GPU-agnostic?

gpu and gpu_device absent
Empty / missing tested list is OK

No

Skip this profile

Target does not match, or
only a GPU name is supplied.
Other profiles may still qualify.

Yes

Yes

Yes

After all profiles

5 Select a profile and return this entry

Choose the best surviving baseline

Keep fitting profiles first; if none fit, retain a baseline for an explicitly estimated GPU count.
Prefer direct > tested > GPU-agnostic. Within the tier: lowest minimum, then profile ID.

Baseline selected

Cannot size this entry

200 recommended

profileId, precision, inferenceBackend and published minimum
recommendation: allocation + warning; estimated count if oversized

200 no_recommendation

Missing capacity, no eligible minimum or no matching profile.
Return a clear reason; no recommendation object.

A nonempty tested list is evidence, not an exclusive allowlist. Untargeted profiles may still be GPU-agnostic.
HTTP 200 can contain both recommended and no_recommendation entries. Empty inventory returns results: [] without an NGC lookup.

Allocation policy: minimum to resource request

After profile selection | Sharing cutoff defaults to 90% | Examples use decimal 80 GB capacity

1 Start with the selected profile and reported capacity

Published minimum

Use the selected profile as published.
Explicit TP=1 and PP=1 baseline.
No invented extra headroom.

Reported device capacity

Use per-device bytes from inventory.
Positive capacity is required.
Not free memory or project quota.

One sharing-policy value

WHOLE_GPU_CUTOFF_PERCENT
Default 90; valid range 1 through 100.
Invalid configuration is rejected.

2 Convert units and apply the cutoff

Compute in bytes without changing the published minimum

m = ceil(published minimum GB x 1,000,000,000); C = reported capacity bytes.
Whole-GPU threshold = ceil(C x cutoffPercent / 100). Compare m with that threshold and C.

3 Return one of three allocations

These ranges show the default 90% policy

Below 90% of capacity

allocationType: gpu_memory; count: 1
gpuMemoryPerDeviceBytes = m
estimated: false

90% through 100%

allocationType: gpu_count; count: 1
gpuMemoryPerDeviceBytes = C
estimated: false

Above device capacity

gpu_count = ceil(m / C)
No per-device memory allocation.
estimated: true + count warning

Example: 71.9 / 80 GB

Below the 72 GB threshold.
Request 71.9 GB of GPU memory.

Example: 72 / 80 GB

Exactly at the 90% threshold.
Request one whole GPU (80 GB).

Example: 81 / 80 GB

Requirement exceeds one device.
Estimate two GPUs; TP is not validated.

Meaning and limits

The cutoff is a sharing rule, not OOM protection. Numerical allocation does not establish runtime compatibility.
Estimated counts do not verify device availability, a supported tensor-parallel configuration or schedulability.
Upstream says minima include runtime overhead and baseline KV cache; exact settings await written confirmation.

Blue: processing Green: successful result Amber: supported alternative / estimate Red: failure Gray: context